



The Standard Every Project Must Meet

Engineering standards, code quality requirements, security practices, performance benchmarks, and delivery protocols applied to every Modulifyr project — no exceptions, no tiers of quality.

CONTENTS

Technical Briefing

01 Overview

02 Six Pillars of Engineering Quality

03 Performance Benchmarks

04 Modular Architecture Principles

05 Delivery Process

06 IP & Ownership Policy

07 Seeing This in Practice

Not Aspirational — Minimum

These are not guidelines we aim for. They are the minimum requirements applied to every codebase we ship, verified on every project during code review, QA, and pre-launch audit — from a \$300 automation script to a multi-month platform build.

This document exists so that anyone evaluating Modulifyr — a technical stakeholder, a procurement team, or an engineer inheriting our code later — has a specific, checkable standard to hold us to. Nothing in here is marketing language; every line below is verified during our own internal review process before a project is considered done.

Six Pillars of Engineering Quality

Code Quality

- ✓ All code reviewed by a senior engineer before merging
- ✓ TypeScript enforced across all JavaScript/Node.js projects
- ✓ ESLint + Prettier on every commit via pre-commit hooks
- ✓ Cyclomatic complexity limits enforced via static analysis
- ✓ 80% unit test coverage minimum, 60% integration coverage
- ✓ No direct console.log in production — structured logging only

Version Control & CI/CD

- ✓ Trunk-based development with feature flags for large changes
- ✓ All deployments via CI/CD — zero manual production deploys
- ✓ GitHub Actions: lint → test → build → deploy pipeline on every PR
- ✓ Environment parity enforced: dev / staging / production always in sync
- ✓ Automated dependency updates via Dependabot with weekly audits
- ✓ All production releases tagged with semantic versioning

Security Standards

- ✓ OWASP Top 10 review conducted on every production system
- ✓ Secrets managed via environment variables — never hardcoded
- ✓ HTTPS enforced on all endpoints with HSTS headers enabled
- ✓ SQL injection prevention via parameterized queries and ORMs
- ✓ Regular dependency vulnerability scans (npm audit, pip audit)
- ✓ Rate limiting and input sanitization on all public-facing APIs

Documentation Standards

- ✓ All public APIs documented with OpenAPI 3.0 specification
- ✓ Architecture Decision Records (ADRs) for all major technical decisions
- ✓ README files include local dev setup, env vars, and deploy instructions
- ✓ Data flow diagrams required for all systems handling PII
- ✓ Runbooks for all production incident and recovery scenarios
- ✓ Inline code comments required for all non-obvious logic

Testing Protocol

- ✓ Unit tests for all business logic with mocked dependencies
- ✓ Integration tests for all API routes and database interactions
- ✓ End-to-end tests for all critical user flows before each release
- ✓ Performance testing on every major feature against SLA thresholds
- ✓ Security penetration testing before every production launch
- ✓ Regression suite run on every pull request via automated CI

Infrastructure Standards

- ✓ Infrastructure-as-code via Terraform for all cloud resources
- ✓ Containerized workloads with Docker for environment consistency
- ✓ Auto-scaling policies configured before production go-live
- ✓ Database backups automated at minimum every 24 hours
- ✓ Disaster recovery procedures documented and tested quarterly
- ✓ Uptime monitoring and alerting configured on day one of production

03 — TARGETS

Performance Benchmarks

Every system we ship is benchmarked against these targets before production go-live. SLA agreements reference these metrics explicitly — not as aspirations, but as contract terms.

METRIC	TARGET	MEASUREMENT
Core Web Vitals (LCP)	< 2.5s	Lighthouse
API Response Time (p95)	< 300ms	Production APM
Time to First Byte	< 200ms	Server metrics
Database Query (p95)	< 100ms	Query profiling
Production Uptime SLA	99.9%	30-day rolling
Error Rate	< 0.1%	Error tracking

Modular Architecture Principles

Our systems are structured around four layers of responsibility, enforced at the dependency, data, and API boundary level — not just folder organization. For the full methodology behind this, see our companion whitepaper, *Modular Architecture Deep Dive*.

#	LAYER
01	Presentation Layer — UI components organized by domain. Server Components for data-fetching; Client Components only for interactivity.
02	Application Layer — Use-case handlers, validation, and orchestration logic. No direct database access. Pure functions where possible.
03	Domain Layer — Business entities, rules, and value objects. Zero external dependencies. 100% unit testable.
04	Infrastructure Layer — Database adapters, API clients, email providers. Swappable implementations behind interface contracts.

Delivery Process

Every engagement, regardless of package, moves through the same five stages:

STAGE	WHAT HAPPENS
Discovery	Understanding operations, workflows, and requirements before any commitment to a build.
Architecture	Designing system structure, modules, and roadmap — the blueprint the build phase executes against.
Development	Building secure, maintainable systems against the six pillars above, with senior review on every merge.
Deployment	Cloud deployment, infrastructure provisioning, and pre-launch testing against the benchmarks in Section 3.
Support	Maintenance, upgrades, and continuous improvement post-launch, scoped separately per engagement.

IP & Ownership Policy

Upon full payment of agreed milestones, clients receive 100% ownership of all custom code, designs, and deliverables produced for their project.

- ✓ Full source code handoff at each milestone
- ✓ No Modulifyr branding in client deliverables
- ✓ IP rights confirmed in signed SOW
- ✓ Proprietary frameworks licensed separately if used

Per our payment milestone policy, deploy access, DNS records, source repository ownership, and final files are transferred only once the corresponding milestone has cleared — this is standard across every package, not a Modulifyr-specific restriction, and is disclosed upfront in every SOW.

07 — Seeing This in Practice

Request an anonymized code sample or schedule an architecture walkthrough with the engineer who would actually work on your project — not a sales handoff.

Modulifyr Engineering · Birtamode, Jhapa, Nepal
contact@modulifyr.com · modulifyr.com/request-proposal